

ThinQ Workspace Sentinel · 팀 온보딩 가이드

LG DX School 5기 · 5분 대기조 · 2026.05.19 ~ 06.25 · 발행 2026-06-01 · v3 친절판 (전문용어 다 풀어씀)

0. 이 문서가 뭐고 왜 봐야 하나

한 줄 요약: 5명이 6주 동안 같이 코딩하는데, 이 문서대로만 따라 하면 처음 보는 사람도 첫날에 셋업하고 첫 PR 올릴 수 있게 만든 안내서.

누가 봐야 하나

5분 대기조 5명 전원 (처음 합류한 사람 포함)

코드를 처음 만지는 신입~중급 개발자 기준으로 설명함

1. 우리가 뭘 만드는 거야?

한 문장: 요양병원에서 RSV·인플루엔자·노로 같은 감염병이 터질 것 같으면 3주 전에 미리 알려주고, LG ThinQ 가전 8종(공기청정기·에어컨·환기·가습·제습·보일러·로봇청소기·스타일러)이 알아서 환경을 바꿔주는 시스템.

왜 만드나?

요양병원에서 노인 한 명 RSV 걸리면 줄줄이 죽음 → 의료법 36조에서 감염관리 의무화

적정성평가(건보공단) 점수 = 병원 수가 직결 = 돈

우리 시스템 도입 시 자동 증빙 + 환경 자동 제어로 점수 가산

2. 처음 보는 용어 사전 (계속 옆에 두고 봐)

협업 관련

GitHub	코드를 인터넷에 올려놓고 같이 작업하는 곳. 카카오톡 단체방처럼 모두가 같은 내용 본다.
레포(repo)	GitHub에 올린 프로젝트 하나. 폴더 같은 거. 우리 레포 = thinq-workspace-sentinel.
브랜치(branch)	본체 코드를 건드리지 않고 따로 작업하는 "사본". 망쳐도 본체에 영향 없음.
커밋(commit)	작업 한 단위를 기록함. "이만큼 했어요" 도장 찍기.
푸시(push)	내 컴퓨터에 있는 커밋을 GitHub로 올리기. 카톡 전송과 비슷.
PR (Pull Request)	"내 작업 봐줘, 본체에 합쳐줘"라고 요청하기. 작업 완료 보고서 같은 거.
머지(merge)	PR을 본체에 합치기. 결재 완료 도장.
리뷰(review)	다른 사람 PR 보고 OK 또는 수정 요청. 서로의 코드 봐주기.

기술 관련

Docker	필요한 프로그램 묶음을 종이상자에 담아놓고 한 번에 켜는 도구. 누구 컴이든 똑같이 동작.
Docker Desktop	윈도우/맥에서 Docker를 클릭으로 켜는 앱. 본인 PC에 깔아야 함.
컨테이너	Docker로 켜 하나의 프로그램 상자. 우리는 컨테이너 4개 정도 같이 돈다.
TimescaleDB	시간순 데이터(센서값) 빠르게 저장·조회하는 DB. PostgreSQL의 친척.
Redis	자주 쓰는 값을 빨리 꺼내려고 두는 임시 메모리 저장소.
FastAPI	파이썬으로 백엔드 서버 만드는 도구.
Next.js	자바스크립트로 웹사이트 만드는 도구.
SSE	서버가 브라우저로 실시간 내용 계속 밀어 보내는 기술. 라이브 스코어 같은 거.
CI	코드 올릴 때마다 자동으로 검사. PR 올리면 GitHub가 자동 테스트해줌.
CD	main에 합쳐지면 자동으로 서버에 올림. 우리는 VM에 자동 배포.
VM	클라우드에서 빌린 컴퓨터. 우리는 GCP에서 uis-capstone 1대 빌려 씬.
API 엔드포인트	서버에서 받을 수 있는 주소 1개. 예: /health = 건강체크.

도메인 관련

REHVA	유럽 환기·공조 표준 협회. 코로나 때 환기율 가이드 만든 곳.
Wells-Riley	공기 중 감염 확률 계산 공식. 1955년부터 쓰는 검증된 식.
Pol	감염 확률 (Probability of Infection). 0~100% 사이. 우리 시스템 핵심 지표.
5-Tier	위험도 5단계: 안전()·주의()·경보()·고위험()·심각(). 신호등 같은 거.
ACH	시간당 환기 횟수. 1시간에 공기가 몇 번 통째로 갈리는지. 의료기관 권장 ACH 6+.
quanta	감염력 단위. 1 quanta = 50% 확률로 감염시키는 양.
KOWAS	질병관리청 하수 기반 감염병 감시 시스템. 우리가 받는 외부 신호 1번.
UIS	다른 캡스톤 팀이 만드는 외부 데이터 시스템. KOWAS·DataLab·OTC 정제 데이터 줌. 우리 X, 다른 팀.
ICN	감염관리 간호사 (Infection Control Nurse). 요양병원 메인 사용자.

3. 우리가 일하는 방식 (한 번에 그림으로)

[본인 PC] [GitHub] [VM (서버)]
↓ 코드 작성 ↓ PR · 리뷰 ↓ 자동 배포
로컬에서 Docker로 자동 검사 (CI) main이 합쳐지면
미니 서버 켜고 테스트 → develop 합침 GitHub가 SSH로
→ main PR VM 들어가서 갱신
→ 합치면 자동 배포

핵심 3가지

코드는 본인 PC에서 짤다. VM에 직접 접속해서 코드 고치면 안 됨 (다음 배포에 다 덮어쓰임).

작업은 항상 본인 브랜치에서. 본체(main·develop)에 직접 푸시 금지.

PR 올리면 자동 검사(CI) 돌아감. 빨간 불 뜨면 고쳐야 합칠 수 있음.

4. 처음 PC에 깔아야 할 것 (15분)

항목	설명	받는 곳
Git	코드 버전관리 기본 도구	Mac: brew install git / Win: git-scm.com
Docker Desktop	미니 서버 한 번에 켜는 도구. 필수	docker.com/products/docker-desktop
Node 20+	Frontend 작업자만. JS 실행 환경	nodejs.org (LTS)
Python 3.11	Backend·ML·Pipeline 작업자만	python.org (3.11.x)
IDE	코드 에디터. VSCode 추천. 자유	code.visualstudio.com
GitHub 계정	본인 ID. 박진영에게 알려서 초대받기	github.com

설치 확인: git --version (2.x+) · docker --version (24+) · node --version (v20+) · python --version (3.11.x)

5. 첫날 셋업 (실제 명령 다 있음)

Step 1. 코드 받기

```
cd ~/projects
git clone https://github.com/zln02/thinq-workspace-sentinel.git
cd thinq-workspace-sentinel
```

→ 우리 프로젝트 전체가 본인 PC에 복사됨.

Step 2. 본인 작업용 브랜치 만들기

```
git checkout develop
git pull
git checkout -b feature/〈모듈〉-〈무엇할지〉-w3
```

예: feature/backend-sse-by-space-w3 · feature/frontend-room-detail-w3

→ 브랜치 이름은 영어·하이픈만. 한글·공백·이름 X.

Step 3. 미니 서버 켜기 (Docker)

```
docker compose -f infra/docker-compose.dev.yml up -d
```

→ Docker Desktop 앱이 미리 켜져 있어야 함. 30초 기다리면 컨테이너 3개 자동 켜짐 (DB·Redis·API).

Step 4. 잘 켜졌는지 확인

```
curl http://127.0.0.1:8003/health
```

→ up 이 보이면 OK. 안 보이면 Step 3 다시.

Step 5. 시뮬레이션 한 번 돌려보기

```
curl -X POST http://127.0.0.1:8003/api/v1/simulate \
-H "Content-Type: application/json" \
-d '{"scenario":"summer_norovirus","minutes":60}'
```

→ 결과로 Poi 30% → 0.01% 같은 게 나오면 모든 게 정상.

Step 6. PC 끌 때

```
docker compose -f infra/docker-compose.dev.yml down
```

→ 컨테이너 다 꺼짐. 데이터는 디스크에 남아있어서 다음에 up -d 하면 그대로.

6. 첫 PR 만들기 (제일 중요한 부분)

Step 1. 코드 수정

본인 IDE(VSCode 등)에서 자기 모듈 파일 수정. 예: Backend 담당이 backend/services/rehva.py 새 파일 작성.

Step 2. 변경사항 저장 (커밋)

```
git status # 어떤 파일 변경?  
git add -A # 모든 변경 추가  
git commit -m "feat(backend): REHVA 분리"
```

커밋 메시지 prefix: feat (새 기능) · fix (버그) · chore (잡일) · docs (문서) · refactor · test

Step 3. GitHub로 보내기 (푸시)

```
git push -u origin feature/〈내브랜치〉
```

→ 처음 푸시할 때만 -u origin 붙임. 두 번째부터는 git push 만.

Step 4. PR 만들기

옵션 A (추천) — 푸시 후 터미널에 뜨는 URL 클릭 → GitHub 웹에서 "Compare & pull request"

옵션 B — 명령어: gh pr create --base develop --web

Step 5. PR 템플릿 4섹션 작성

```
어떤 모듈: backend  
왜: REHVA를 백엔드 서비스로 분리. 프론트에서도 호출 가능.  
어떻게 검증: [x] curl /health OK · [x] 단위 테스트 추가 · [x] 시물 동일  
UIS 경계: [x] UIS 미수정
```

Step 6. 자동 검사(CI) 통과 기다리기

PR 페이지 아래 동그라미 3개:

```
python — 파이썬 코드 검사  
frontend — 프론트 빌드 (Frontend 작업 시)  
secret-scan — 비밀번호 누출 검사
```

→ 다 이면 OK. 이면 클릭해서 로그 보고 수정.

Step 7. 리뷰 받기 → Step 8. 머지

리뷰어 = 박진영 또는 다른 모듈 담당자. "Approve" 누르면 머지 가능 → "Merge pull request" 클릭 → 끝.

7. 만약 문제 생기면 (자주 있는 5가지)

A. Docker Desktop 안 켜짐

증상: Cannot connect to Docker daemon

해결: Docker Desktop 앱 직접 실행 → 고래 아이콘 안정될 때까지 30초 기다림

B. 8003 포트 이미 사용 중

증상: port is already allocated

해결: 다른 앱이 쓰는 중. 끄거나 docker-compose.dev.yml 에서 포트 변경

C. CI 빨간 불

증상: python failed

해결: 클릭 → 로그 → 본인 PC에서 ruff check backend pipeline 재현 → 수정 → push

D. develop 충돌

증상: branch has conflicts

해결: git checkout develop && git pull → git checkout feature/... → git rebase develop → 충돌 해결 → git push --force-with-lease

E. 다른 사람 머지 후 최신 코드

해결: git checkout develop && git pull

8. 6주 일정

주차	다 같이 해야 할 것
W3 6/2~8	외부 신호(KOWAS) → 예측(XGBoost) → 시물 통합. 백엔드·파이프라인 메인.
W4 6/9~15	화면 만들기 + 75초 시연 영상. 프론트 메인.
W5 6/16~22	다층 검증·자동 해제·모바일 화면
W6 6/23~25	최종 발표 + 시연 영상 녹화

9. 역할 분담

〈〉 자리는 박진영(Tech Lead)이 실제 멤버 이름으로 채울 예정.

모듈	담당	주로 뭐 함
ml/	박진영 Tech Lead · ML	머신러닝 모델 (XGBoost·TFT) · 감염 확률 계산
backend/	〈Backend 담당〉	서버 (FastAPI) · 가전 제어 · DB 쿼리
pipeline/	〈Data 담당〉	UIS에서 외부 데이터 받기 · 시뮬레이터
frontend/	〈Frontend 담당〉	화면 (Next.js) · 병실 그리드 · 시연 모드
infra/	〈DevOps/QA 담당〉	자동 검사·배포 · 모니터링 · 부하 테스트

10. 절대 하면 안 되는 5가지

1. main · develop 브랜치에 직접 푸시 금지 — 항상 PR 통해서만
2. .env 파일 commit 금지 — 비밀번호·API 키 들어있음
3. VM에 직접 SSH 접속해서 코드 수정 금지 — 다음 배포에 덮어쓰여 작업 손실
4. ~/urban-immune-system/ 디렉토리 수정 금지 — 다른 캡스톤 팀(UIS) 영역
5. public.* 스키마(DB) 수정 금지 — UIS 팀 테이블. 우리는 sentinel.* 만

11. 어디서 도움 받나

막혔을 때	어디로
Git/PR/CI 등 협업 도구	Slack #sentinel-dev + 박진영 DM
본인 모듈 코드	Slack #sentinel-dev 채널에서 질문
다른 모듈 코드 보달라	해당 담당자에게 PR 리뷰 요청
시스템 전체 설계	박진영(Tech Lead)

24h 안에 막혔으면 무조건 박진영에게 DM — 혼자 끄공 앓지 말 것.

12. UIS와 우리 관계 (헛갈리니까 다시)

UIS (다른 캡스톤 팀) — 데이터 — 5분 대기조 (우리)
~/urban-immune-system/ 공급 ~/thing-workspace-sentinel/

외부 데이터 수집 · 정제 요양병원 감염 예측 + ThinQ
- KOWAS 하수 - REHVA Wells-Riley
- DataLab 검색 트렌드 - XGBoost 5-Tier
- OTC 약국 판매 - ThinQ 가전 8종

DB: public.* 스키마 DB: sentinel.* 스키마
UIS 팀 소유 · 절대 건드리지 X 우리 소유 · 자유롭게 작업

같은 VM에 살지만, 우리 작업은 완전히 분리된 폴더·DB 스키마 안에서만 한다.

부록. 한 줄 명령어 모음

```
# 처음 셋업
git clone https://github.com/zln02/thinq-workspace-sentinel.git
cd thinq-workspace-sentinel
docker compose -f infra/docker-compose.dev.yml up -d
curl http://127.0.0.1:8003/health

# 매일 시작할 때
git checkout develop && git pull
git checkout -b feature/〈모듈〉-〈할일〉-w3

# 작업 끝나면
git add -A
git commit -m "feat(〈모듈〉): 〈한 줄〉"
git push -u origin feature/〈내브랜치〉
gh pr create --base develop --web

# PC 끌 때
docker compose -f infra/docker-compose.dev.yml down
```

ThinQ Workspace Sentinel · 박진영 (Tech Lead) · v3 친절판 · GitHub github.com/zln02/thinq-workspace-sentinel (private) · 변경은 docs/team_onboarding_guide.md 기준